DevTrack V2 — System Architecture Document

Status: Planning Phase | **Version:** 0.2 (Free Tier Edition) | **Author:** Staff Engineering

---

1. Architectural Philosophy

Core Tenets

- **Domain isolation over DRY**: Two slightly similar things in different domains should NOT share code. Shared code creates invisible coupling. Prefer duplication across domains, abstraction within them.
- **Async by default**: Any operation touching GitHub API or AI inference runs via scheduled jobs or async handlers — never blocking an HTTP request.
- **Fail gracefully**: AI features degrade, not crash. GitHub rate limits are expected, not exceptional.
- **Schema-first**: Database schema is source of truth. Types flow from schema upward to API to frontend.
- **Upgrade-ready from day 1**: Free-tier constraints are handled at the infrastructure/transport layer only. Domain logic is written as if BullMQ exists — swapping it in later requires only wiring changes, not rewrites.
- **Observability-ready from day 1**: Structured logging, trace IDs, metric hooks built in from the start.

---

2. Free Tier Stack (Zero Cost)

Layer	Service	Free Limits	Notes
Frontend	Vercel Hobby	100GB bandwidth, 100GB-hrs compute	Personal/non-commercial use
Backend API	Render Web Service	750 instance-hrs/month, sleeps after 15min	~1min cold start
Database	Neon Postgres	0.5GB storage, 100 CU-hrs/month	Never expires, no card, commercial OK
Redis/Queue	❌ Not free	—	Replaced with in-process scheduler
Background Workers	❌ Not free on Render	—	Merged into API process

Layer	Service	Free Limits	Notes
Auth	Clerk	50,000 monthly retained users	GitHub OAuth included

Key Constraints & How We Handle Them

No BullMQ/Redis → `@nestjs/schedule` (NestJS built-in cron). All job logic written in isolated service classes — extracting to BullMQ later is a transport-layer change only, zero domain logic rewrites.

Render cold starts → Acceptable during development. Accept the ~1min first-request delay until you go paid ($7/mo Starter removes it).

Neon 0.5GB storage → Sufficient for MVP (~500 users, see storage budget in Section 5). Monitor with `SELECT pg_database_size(current_database())`. Upgrade to Neon Launch ($19/mo) when approaching the limit.

Neon 100 CU-hrs/month → Scale-to-zero means idle time is free. 100 CU-hrs covers active development well. Hard cutoff if limit hit — database pauses until next billing cycle reset.

---

3. Monorepo Structure

```
devtrack-v2/
├── apps/
│   ├── web/                        # Next.js 15 → Vercel (free Hobby)
│   └── api/                        # NestJS → Render web service (free)
│       └── src/modules/jobs/       # @nestjs/schedule (replaces BullMQ workers)
│
├── packages/
│   ├── database/                   # Prisma schema + Neon Postgres
│   ├── types/                      # Shared TypeScript interfaces (zero deps)
│   ├── config/                     # Zod env validation
│   ├── ui/                         # shadcn/ui design system
│   ├── ai-client/                  # Provider abstraction (NVIDIA NIM / Groq)
│   └── logger/                     # Pino structured logger
│
├── infra/
│   ├── docker/
│   ├── compose/
│   │   └── docker-compose.local.yml  # Local: Postgres + Redis for full-fidelity dev
│   └── scripts/
│       └── db-backup.sh            # Optional manual Neon backup
│
├── docs/
│   ├── architecture/               # ADRs
```

```
│   ├── api/                          # OpenAPI specs
│   └── runbooks/
│
├── turbo.json
├── pnpm-workspace.yaml
└── package.json
```

Why No `apps/workers/`?

Render background worker service type requires a paid plan. Workers are merged into `apps/api/src/modules/jobs/`. Structured identically to a standalone workers app — same service classes, same interfaces. When paid: create `apps/workers/`, move job classes there, add BullMQ as transport. Zero domain logic changes.

Local Development

Run Postgres and Redis locally via Docker for full-fidelity dev. Local env has BullMQ available — only the deployed free environment uses the scheduler fallback.

yaml

```yaml
# infra/compose/docker-compose.local.yml
services:
  postgres:
    image: postgres:16-alpine
    ports: ["5432:5432"]
    environment:
      POSTGRES_DB: devtrack
      POSTGRES_PASSWORD: devtrack
  redis:
    image: redis:7-alpine
    ports: ["6379:6379"]
```

---

4. Backend Domain Architecture (NestJS)

Feature-based domain modules — not layer-based (not controllers/services/repos as top-level folders).

```
apps/api/src/
├── main.ts
├── app.module.ts
│
├── modules/
│   ├── auth/                        # Clerk JWT verification + guards
│   │   ├── auth.module.ts
│   │   ├── auth.service.ts
│   │   ├── guards/                  # AuthGuard, RolesGuard, SubscriptionGuard
│   │   └── decorators/              # @CurrentUser()
│   │
│   ├── users/                       # User profile & preferences
│   ├── github/                      # GitHub API calls + rate limit management
│   ├── analytics/                   # Streak + stat computation
│   │
│   ├── ai/                          # AI orchestration layer
│   │   ├── providers/               # NVIDIA NIM, Groq, Mock adapters
│   │   ├── prompts/                 # Versioned prompt templates (v1/, v2/)
│   │   └── pipelines/               # Multi-step AI workflows
│   │
│   ├── projects/                    # Project + task management
│   ├── learning/                    # Learning logs + streak engine
│   ├── profiles/                    # Public developer profiles
│   ├── subscriptions/               # Feature gating (future paid tiers)
│   │
│   └── jobs/                        # Replaces standalone workers app on free
tier
│       ├── jobs.module.ts
│       ├── github-sync.job.ts       # @Cron('0 2 * * *') nightly sync
│       ├── ai-analysis.job.ts       # @Cron('0 3 * * *') nightly AI analysis
│       ├── streak-compute.job.ts    # @Cron('0 0 * * *') daily streak update
│       └── insight-gen.job.ts       # @Cron('0 4 * * 1') weekly insights
│
├── infrastructure/
│   ├── database/                    # PrismaService singleton (Neon connection)
│   ├── cache/                       # In-memory cache via @nestjs/cache-manager
│   ├── http/                        # Axios instances + interceptors
│   └── events/                      # Internal EventEmitter2 bus
│
└── common/
    ├── decorators/
    ├── filters/                     # Global exception filters
    ├── interceptors/                # Logging, transform
    └── pipes/
```

Jobs Module — BullMQ-Ready Pattern

Each job is a standalone service class with dependencies injected. The `@Cron()` decorator is the only thing that changes when migrating to BullMQ — the business logic inside is identical:

typescript

```typescript
@Injectable()
export class GitHubSyncJob {
  constructor(
    private readonly githubService: GitHubService,
    private readonly analyticsService: AnalyticsService,
    private readonly logger: Logger,
  ) {}

  // Free tier: cron trigger
  @Cron('0 2 * * *')
  async handleNightlySync() {
    const users = await this.getActiveUsers();
    for (const user of users) {
      await this.syncUser(user.id); // sequential, rate-limit safe
    }
  }

  // This exact method becomes the BullMQ process() handler when upgrading
  async syncUser(userId: string): Promise<void> {
    await this.githubService.syncRepos(userId);
    await this.analyticsService.recompute(userId);
  }
}
// Upgrade path: wrap syncUser() in BullMQ processor, replace @Cron with producer.
// Business logic inside syncUser() is untouched.
```

---

5. Database Architecture (Neon Postgres + Prisma)

Why Neon over Render Postgres?

- **Never expires** (Render free deletes after 30 days)
- No credit card required, commercial use allowed
- Serverless scale-to-zero (idle costs zero CU-hours)
- Database branching — create a branch per PR (free plan supports this)
- Prisma official support with connection pooling via pgBouncer

Neon + Prisma Connection Setup

prisma

```
// packages/database/prisma/schema.prisma
datasource db {
  provider  = "postgresql"
  url       = env("DATABASE_URL")          // pooled — for runtime queries
  directUrl = env("DATABASE_URL_UNPOOLED") // direct — for prisma migrate only
}
```

env

```
DATABASE_URL="postgresql://user:pass@ep-xxx.neon.tech/devtrack?pgbouncer=true&connec
DATABASE_URL_UNPOOLED="postgresql://user:pass@ep-xxx.neon.tech/devtrack"
```

Core Schema

prisma

```
// === IDENTITY & AUTH ===

model User {
  id              String           @id @default(uuid())
  email           String           @unique
  username        String           @unique
  display_name    String?
  avatar_url      String?
  timezone        String           @default("UTC")
  subscription    Subscription?
  github_account GitHubAccount?
  profile         DeveloperProfile?
  learning_logs   LearningLog[]
  projects        Project[]
  audit_logs      AuditLog[]
  created_at      DateTime         @default(now())
  updated_at      DateTime         @updatedAt
  deleted_at      DateTime?
}

model GitHubAccount {
  id              String      @id @default(uuid())
  user_id         String      @unique
  user            User        @relation(fields: [user_id], references: [id])
  github_id       Int         @unique
  github_login    String      @unique
  access_token    String      // AES-256-GCM encrypted at rest
  token_scope     String[]
  last_synced_at DateTime?
  sync_status     SyncStatus  @default(PENDING)
  repos           GitHubRepo[]
  created_at      DateTime    @default(now())
  updated_at      DateTime    @updatedAt
}

// === GITHUB INGESTED DATA ===

model GitHubRepo {
  id              String        @id @default(uuid())
  account_id      String
  account         GitHubAccount @relation(fields: [account_id], references: [id])
  github_repo_id Int
  name            String
  full_name       String
  description     String?
  language        String?
```

```
  is_private     Boolean
  stars          Int           @default(0)
  forks          Int           @default(0)
  last_pushed_at DateTime?
  analysis       RepoAnalysis?
  commit_stats   CommitStat[]
  created_at     DateTime      @default(now())
  updated_at     DateTime      @updatedAt

  @@unique([account_id, github_repo_id])
  @@index([account_id])
}

model CommitStat {
  id         String     @id @default(uuid())
  repo_id    String
  repo       GitHubRepo @relation(fields: [repo_id], references: [id])
  date       DateTime   @db.Date       // aggregated per day — not per commit
  count      Int
  additions  Int        @default(0)
  deletions  Int        @default(0)
  created_at DateTime   @default(now())

  @@unique([repo_id, date])
  @@index([repo_id, date])
}

// === AI ANALYSIS ===

model RepoAnalysis {
  id                 String     @id @default(uuid())
  repo_id            String     @unique
  repo               GitHubRepo @relation(fields: [repo_id], references: [id])
  vulnerabilities    Json
  complexity_hotspots Json
  next_steps         Json
  security_score     Int?
  model_used         String
  prompt_version     String
  analyzed_at        DateTime   @default(now())
  expires_at         DateTime   // re-analyzed after 7 days
}

model AIInsight {
  id            String      @id @default(uuid())
  user_id       String
  insight_type  InsightType
```

```
  content        Json
  model_used     String
  prompt_version String
  is_read        Boolean     @default(false)
  generated_at   DateTime    @default(now())

  @@index([user_id, insight_type])
  @@index([user_id, is_read])
}

// === LEARNING ===

model LearningLog {
  id            String   @id @default(uuid())
  user_id       String
  user          User     @relation(fields: [user_id], references: [id])
  date          DateTime @db.Date
  duration_mins Int
  topic         String
  notes         String?
  mood          Mood
  tags          String[]
  created_at    DateTime @default(now())
  updated_at    DateTime @updatedAt

  @@unique([user_id, date])
  @@index([user_id, date])
}

model LearningStreak {
  id               String    @id @default(uuid())
  user_id          String    @unique
  current_streak   Int       @default(0)
  longest_streak   Int       @default(0)
  last_active_date DateTime? @db.Date
  updated_at       DateTime  @updatedAt
}

// === PROJECTS ===

model Project {
  id             String        @id @default(uuid())
  user_id        String
  user           User          @relation(fields: [user_id], references: [id])
  github_repo_id String?
  title          String
  description    String?
```

```
  status         ProjectStatus @default(ACTIVE)
  visibility     Visibility    @default(PRIVATE)
  tech_stack     String[]
  tasks          Task[]
  created_at     DateTime      @default(now())
  updated_at     DateTime      @updatedAt
  deleted_at     DateTime?

  @@index([user_id, status])
}

model Task {
  id         String     @id @default(uuid())
  project_id String
  project    Project    @relation(fields: [project_id], references: [id])
  title      String
  status     TaskStatus @default(TODO)
  priority   Priority   @default(MEDIUM)
  due_date   DateTime?
  created_at DateTime   @default(now())
  updated_at DateTime   @updatedAt

  @@index([project_id, status])
}

// === PROFILES & SUBSCRIPTIONS ===

model DeveloperProfile {
  id         String   @id @default(uuid())
  user_id    String   @unique
  user       User     @relation(fields: [user_id], references: [id])
  slug       String   @unique  // public URL: devtrack.io/u/vikash
  bio        String?
  headline   String?
  skills     String[]
  is_public  Boolean  @default(false)
  view_count Int      @default(0)
  created_at DateTime @default(now())
  updated_at DateTime @updatedAt
}

model Subscription {
  id                 String    @id @default(uuid())
  user_id            String    @unique
  user               User      @relation(fields: [user_id], references: [id])
  plan               Plan      @default(FREE)
  status             SubStatus @default(ACTIVE)
```

```
  current_period_end DateTime?
  created_at         DateTime  @default(now())
  updated_at         DateTime  @updatedAt
}

// === AUDIT ===

model AuditLog {
  id          String   @id @default(uuid())
  user_id     String?
  user        User?    @relation(fields: [user_id], references: [id])
  action      String
  entity_type String?
  entity_id   String?
  metadata    Json?
  ip_address  String?
  created_at  DateTime @default(now())

  @@index([user_id])
  @@index([created_at])
}

// === ENUMS ===

enum SyncStatus    { PENDING SYNCING COMPLETED FAILED }
enum InsightType   { WEEKLY_SUMMARY REPO_ANALYSIS GROWTH_REPORT PROJECT_IDEA }
enum Mood          { GREAT GOOD NEUTRAL TIRED BURNED_OUT }
enum ProjectStatus { ACTIVE PAUSED COMPLETED ARCHIVED }
enum TaskStatus    { TODO IN_PROGRESS IN_REVIEW DONE }
enum Priority      { LOW MEDIUM HIGH CRITICAL }
enum Visibility    { PRIVATE PUBLIC }
enum Plan          { FREE PRO TEAM }
enum SubStatus     { ACTIVE CANCELED PAST_DUE }
```

Storage Budget (0.5GB Free Tier)

Table	Est. rows (500 users)	Est. size
Users + profiles	500	~1MB
GitHubRepo	5,000	~5MB
CommitStat	150,000	~30MB
LearningLog	10,000	~5MB
RepoAnalysis	1,000	~10MB

Table	Est. rows (500 users)	Est. size
AIInsight	5,000	~15MB
AuditLog	50,000	~20MB
Total		**~86MB — well within 0.5GB**

Upgrade to Neon Launch ($19/mo) when approaching 400MB or 2,000+ active users.

---

6. Authentication Flow

```
User → Clerk (GitHub OAuth) → Clerk session token
  ↓
Frontend sends Clerk JWT on every API request (Authorization: Bearer)
  ↓
NestJS AuthGuard verifies JWT against Clerk JWKS endpoint (cached in-memory)
  ↓
Extracts { userId, email } → attaches to Request object
  ↓
@CurrentUser() decorator available in all controllers
```

Deferred OAuth Scopes (retained from V1)

- Sign-up: `read:user, public_repo`
- Elevated (`repo`): prompted only when user enables private repo tracking
- Granted scopes stored in `GitHubAccount.token_scope[]`

Security

- GitHub access tokens encrypted at rest (AES-256-GCM, key in env)
- Rate limiting via `@nestjs/throttler` with in-memory store (no Redis needed on free)
- RBAC guards: `@Roles()`, `@RequireFeature()`
- Audit log on all security-sensitive mutations

---

7. GitHub Ingestion Pipeline (Free Tier Shape)

No BullMQ. No separate worker process. All async work via `@nestjs/schedule`.

```
Nightly cron (2 AM UTC) → GitHubSyncJob.handleNightlySync()
  ↓
Fetch all users with connected GitHub accounts
  ↓
For each user (sequential, rate-limit safe, 200ms delay between):
  1. GitHub API → fetch repos + commit activity (last 90 days)
  2. Upsert GitHubRepo + CommitStat in Neon
  3. Emit internal event: sync.completed
  ↓
EventEmitter2 → AnalyticsService.recompute(userId)
  → Recalculate streak
  → Recalculate language breakdown
  ↓
If repo changed → inline async AI analysis
  → Fetch README + file tree
  → Groq/NIM inference
  → Store RepoAnalysis (expires_at = now + 7 days)
```

Rate Limit Management

- GitHub: 5000 req/hr per token
- `await sleep(200)` between per-user sync calls
- Check `X-RateLimit-Remaining` header; pause sync if < 100
- On 429: exponential backoff using `retry-after` header

Manual Sync (User-Triggered)

```
POST /github/sync
  → Throttle: max 1 manual sync per hour per user
  → Run syncUser(userId) inline async
  → Return 202 Accepted immediately
GET  /github/sync/status  → poll for completion
```

---

8. AI Orchestration

Provider-agnostic. Never call AI providers directly from domain logic.

```
packages/ai-client/
├── ai-provider.interface.ts      # IAIProvider { complete(), embed() }
├── providers/
│   ├── groq.provider.ts          # Primary on free (30 req/min, 6000 req/day)
│   ├── nvidia-nim.provider.ts    # Secondary (check NIM free limits)
│   ├── gemini.provider.ts        # Fallback (15 RPM, 1M TPD free)
│   └── mock.provider.ts          # Testing/CI
└── factory.ts                    # createAIProvider(config) → IAIProvider
```

Prompt Versioning

Prompts are immutable once shipped. New versions added as new files, never editing existing:

```
apps/api/src/modules/ai/prompts/
├── v1/
│   ├── repo-analysis.prompt.ts
│   ├── weekly-insight.prompt.ts
│   └── project-idea.prompt.ts
└── v2/   ← add here, never edit v1/
```

---

9. Frontend Architecture (Next.js 15)

```
apps/web/
├── app/
│   ├── (public)/
│   │   ├── u/[slug]/             # Public developer profile (SSR)
│   │   ├── login/
│   │   └── page.tsx
│   │
│   ├── (dashboard)/              # Auth required (Clerk middleware)
│   │   ├── layout.tsx
│   │   ├── overview/
│   │   ├── analytics/
│   │   ├── github/
│   │   ├── projects/
│   │   ├── learning/
│   │   ├── insights/
│   │   └── settings/
│   │
│   └── layout.tsx
│
├── components/
│   ├── ui/                       # Re-exported from packages/ui (shadcn)
```

```
|   ├── layout/                  # Shell, Sidebar, Navbar
|   └── features/
|       ├── analytics/
|       ├── github/
|       ├── learning/
|       ├── projects/
|       └── insights/
|
├── lib/
|   ├── api/                     # TanStack Query hooks per domain
|   └── utils/
|
└── middleware.ts                # Clerk auth middleware
```

Data Fetching Strategy

- **Server Components:** initial page render, public profiles (SEO)
- **TanStack Query:** interactive dashboard data, polling sync status
- **Optimistic updates:** task status, learning log saves
- **No client-side token storage:** Clerk handles session entirely

---

10. Deployment Architecture

```
GitHub → GitHub Actions CI
  ↓ on PR
    lint + typecheck (Turborepo cache, parallel)
    unit tests
    Vercel preview deploy (web)

  ↓ on merge to main
    full test suite
    prisma migrate deploy → Neon (direct URL)
    Render deploy (api) via render.yaml
    Vercel deploy (web) — automatic on push

Production:
  Vercel Hobby    → Next.js 15  (global CDN, free)
  Render Free     → NestJS API  (sleeps after 15min, free)
  Neon Free       → Postgres    (never expires, free)
  Clerk Free      → Auth        (50k MAU, free)
```

render.yaml

```yaml
services:
  - type: web
    name: devtrack-api
    runtime: node
    plan: free
    buildCommand: pnpm --filter api build
    startCommand: node apps/api/dist/main.js
    envVars:
      - key: DATABASE_URL
        sync: false
      - key: DATABASE_URL_UNPOOLED
        sync: false
      - key: CLERK_SECRET_KEY
        sync: false
      - key: GROQ_API_KEY
        sync: false
      - key: NVIDIA_NIM_API_KEY
        sync: false
      - key: ENCRYPTION_KEY
        sync: false
      - key: NODE_ENV
        value: production
```

---

11. Observability (Free Tier)

```
Logging:   Pino JSON → Render built-in log viewer (free)
Errors:    Sentry free tier (5k errors/month)
Uptime:    UptimeRobot free (50 monitors, 5-min checks)
DB stats:  Neon dashboard (built-in, free)
```

Every log line: `{ traceId, userId?, module, action, durationMs, level }`

---

12. Upgrade Path (Free → Paid, When Ready)

Pain point	Fix	Cost added
API cold starts (15min idle)	Render Starter for api	+$7/mo
No background workers	`apps/workers/` + BullMQ + Render worker service	+$7/mo

Pain point	Fix	Cost added
No persistent Redis	Render Redis paid	+$10/mo
Neon 0.5GB limit	Neon Launch plan	+$19/mo
Vercel non-commercial	Vercel Pro	+$20/mo
Total when scaling		**~$63/mo**

Domain logic, schemas, API contracts — nothing changes. Only transport layer and infra config.

---

13. MVP vs V2 Roadmap

MVP (Weeks 1–8) — Core Loop

- [ ] Monorepo scaffold (Turborepo + pnpm)
- [ ] CI/CD (GitHub Actions → Vercel + Render)
- [ ] Auth: Clerk + GitHub OAuth
- [ ] Neon Postgres + Prisma migration baseline
- [ ] GitHub sync: nightly `@nestjs/schedule` job
- [ ] Learning log CRUD + streak engine
- [ ] Analytics dashboard (commit graph, language chart, streaks)
- [ ] AI repo analysis (Groq free, one pipeline)
- [ ] Public developer profile (read-only)

V2.1 (Weeks 9–14) — Intelligence

- [ ] Weekly AI insight generation
- [ ] AI project idea generator
- [ ] Learning roadmap generation
- [ ] Prompt A/B versioning framework

V2.2 (Weeks 15–20) — Power Features

- [ ] Kanban project management
- [ ] Subscription tiers + feature gating
- [ ] PDF growth report export
- [ ] GitHub webhooks (real-time sync)
- [ ] Profile themes + portfolio customization

V2.3 (Weeks 21+) — Scale & Collaboration

- [ ] Migrate to BullMQ (paid Render workers)
- [ ] Team workspaces + peer review

- [ ] Vector search on insight history
- [ ] AI mentor chat (full context)
- [ ] Public API v2

---

14. Architecture Decision Records

ADR-001: Neon over Render Postgres

Render free Postgres expires and is hard-deleted after 30 days with no backups. Neon free never expires, commercial use allowed, no credit card, supports database branching per PR, and has official Prisma support. Clear winner.

ADR-002: @nestjs/schedule over BullMQ (free tier)

BullMQ requires persistent Redis. Render free Redis loses all data on restart — queued jobs vanish silently. `@nestjs/schedule` runs inside the API process with zero external dependencies. Job logic is written in isolated service methods so migrating to BullMQ is a transport-layer change only.

ADR-003: Merged workers into api/ (free tier)

Render background worker service type is paid-only. All background logic lives in `apps/api/src/modules/jobs/`. Module is structured identically to a standalone workers app — extraction is straightforward when upgrading.

ADR-004: In-memory rate limiting (@nestjs/throttler)

`@nestjs/throttler` supports in-memory storage. Sufficient for free-tier scale. Upgrade path: swap throttler storage adapter to Redis — one config line change.

ADR-005: Groq as primary AI provider (free tier)

Groq offers 30 req/min and 6,000 req/day free — sufficient for nightly batch analysis of MVP user base. Provider abstraction means swapping to NVIDIA NIM as primary is a config change, not a code change.

ADR-006: NestJS over Express

Built-in DI, modules, `@nestjs/schedule`, guards, interceptors. Critical for a small team maintaining complex domain logic without assembling these patterns from scratch.

ADR-007: Prisma over Drizzle

Prisma's migration system and schema validation are more mature for a complex multi-table schema. Neon's Prisma adapter is officially supported.

ADR-008: pnpm + Turborepo over npm workspaces

Turborepo's build caching makes CI ~80% faster after the first run. pnpm's strict hoisting

prevents phantom dependency bugs common in npm workspaces.